Assignment 7 — Incremental Data Processing with Delta Lake

Delta Lake MERGE · SCD Type 1 & Type 2 · executed 2026-08-03 01:19:49 UTC · engine: delta-rs 1.6.2 (native Delta Lake)

1. Objective and dataset

Build an incremental (batch) data pipeline on Delta Lake: land a customer snapshot in a Delta table, clean it, simulate a next-day change feed, apply a **MERGE** to upsert the changes, validate the result, and publish a final dataset with a business summary. The source is the *Sample - Superstore* order export (9,994 order lines), aggregated into a 793-customer dimension and then deliberately degraded with nulls, duplicate rows and inconsistent strings so that the cleaning step has real work to do.

Input	Rows	Contents
data/customer_master.csv	866	full snapshot: 793 customers + 45 exact duplicates + 25 late-arriving versions + 3 null ke
data/customer_incremental.csv	162	change feed: updates to existing customers, brand-new customers, 5 duplicated source

2. Pipeline

A three-layer medallion design. Every write is a Delta commit, so each table carries an ACID transaction log and can be read at any earlier version.

Layer	Delta table	What it holds
Bronze	bronze_customer_raw	the CSV landed verbatim, every column as STRING
Silver	silver_customer_master	cleaned, typed, de-duplicated — one row per customer
Gold	gold_customer_scd1	SCD Type 1 — current state, one row per customer
Gold	gold_customer_scd2	SCD Type 2 — full history with effective dates

3. Cleaning

One shared function cleans both the master snapshot and the incremental feed, which guarantees they are normalised identically — if they were not, the change-detection hash would report false differences and the merge key could fail to match.

Rule	Effect on the master snapshot
Trim whitespace, collapse spaces, standardise case	'CONSUMER ' and "Consumer" no longer split into two segments
Drop rows with a null business key	3 rows removed
Drop exact duplicate rows	45 rows removed
Keep newest row per customer_id	25 rows removed
Cast to int / float / date / timestamp	enables arithmetic, range filters, schema enforcement
Impute remaining nulls with documented defaults	123 null cells filled

Result: **866 raw rows** → **793 clean rows** (73 removed), zero nulls, zero duplicates, unique primary key. The incremental feed cleaned from 162 to 157 rows (5 duplicated source rows removed). That last step is a hard requirement, not a nicety: **MERGE fails outright if one source row matches the same target row more than once**.

4. The MERGE operations

Before merging, every incoming row is compared against the target using an MD5 hash of the tracked attributes, which gives an exact expectation for what the merge should do:

Classification	Rows	Expected merge action
NEW — key not in target	45	INSERT
CHANGED — key present, hash differs	101	UPDATE (SCD1) / new version (SCD2)
UNCHANGED — key present, hash identical	11	no-op

SCD Type 1 — overwrite in place

```
MERGE INTO gold_customer_scd1 AS t
USING incremental_batch AS s ON t.customer_id = s.customer_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Metric	Value	Expected
source rows	157	157
rows updated	112	101 changed + 11 identical = 112
rows inserted	45	45
table rows	793 → 838	793 + 45
Delta version	0 → 1	one atomic commit

SCD Type 2 — full history

A single MERGE cannot both close an old row and insert its replacement, because one source row can only act on one target row. The standard Delta pattern feeds MERGE a union: **Part A** is every incoming row keyed on customer_id (it matches the current row and closes it), and **Part B** is the changed rows only with **merge_key = NULL** — NULL never equals anything, so those rows are always NOT MATCHED and are inserted as the new version.

```
MERGE INTO gold_customer_scd2 AS t
USING staged_source AS s ON t.customer_id = s.merge_key AND t.is_current = true
WHEN MATCHED AND t.record_hash <> s.record_hash
     THEN UPDATE SET is_current = false, effective_end_date = s.effective_start_date
WHEN NOT MATCHED THEN INSERT *
```

Metric	Value	Expected
staged source rows	157 + 101 = 258	Part A + Part B
versions closed out	101	101
rows inserted	146	101 new versions + 45 new customers
table rows	793 → 939	793 + 101 + 45
current rows	838	one per customer, matches SCD1

5. Validation

31 assertions run against the persisted Delta tables — read back from disk, not from memory — and all pass:

Area	Checks	Result
SCD1 — counts, primary key, nulls, merge metrics	7	all pass

SCD2 — surrogate key, one current row per customer, effective-date integrity, no gaps or overlaps	10	all pass
Cross-table suite — SCD1 vs SCD2 agreement, feed fully applied, domain checks	14	all pass
Time travel — version 0 reproduces the pre-merge state exactly	1	pass

6. Results

	Rows	Distinct customers
customer_master.csv (raw)	866	793
silver_customer_master (clean)	793	793
gold_customer_scd1 (after MERGE)	838	838
gold_customer_scd2 (all versions)	939	838
of which current	838	
of which expired	101	

Final business view: **838 customers**, **$2,589,469.56** lifetime sales, **$316,504.48** profit.

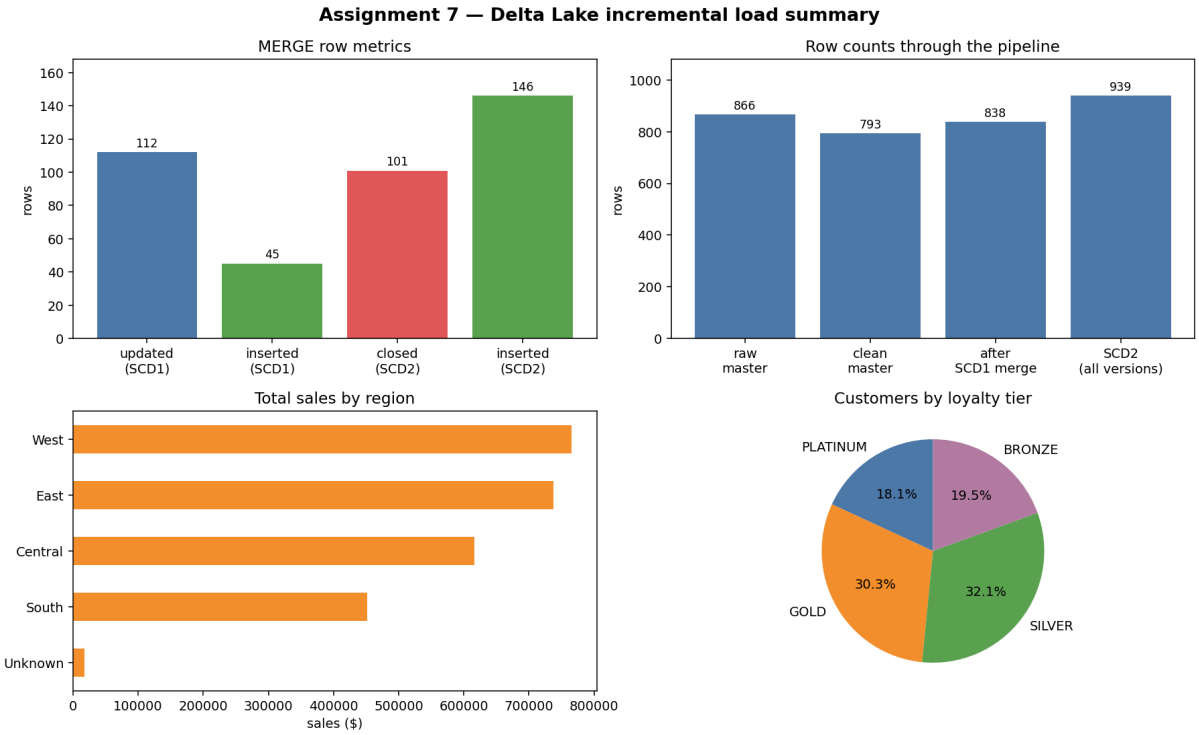

Figure 1 — merge metrics, row counts through the pipeline, sales by region and customer mix by loyalty tier.

7. Discussion

Why MERGE rather than a full overwrite. Rewriting the whole table each batch is simple but wasteful, and it destroys any record of what changed. MERGE rewrites only the affected files, runs as a single ACID transaction, and records row-level metrics in the transaction log. Readers querying during the merge see the previous version, never a half-written one.

Type 1 versus Type 2. Type 1 keeps one row per customer and overwrites it — the right choice when only the current state matters and query simplicity is the priority. Type 2 keeps every version, which is what you need to answer "what was this customer's segment when that order was placed?". The cost is a larger table and a mandatory `WHERE is_current = true` on every current-state query. The record_hash is what makes Type 2 behave: without it, a feed that re-sends identical rows would create a new version for every customer every day. Here it correctly left the 11 unchanged rows alone.

Trade-offs and next steps. These tables are unpartitioned, which is fine at this size but would need partitioning (by region, say) plus periodic OPTIMIZE / Z-ORDER at scale. Surrogate keys are allocated in blocks so the sequence has gaps — harmless, since a surrogate key only has to be unique. A production version would enable Change Data Feed for downstream consumers, schedule VACUUM once the time-travel

retention window has passed, and quarantine bad rows into a rejects table instead of imputing them.

8. Reproducing this run

```
pip install -r requirements.txt
python scripts/generate_datasets.py   # rebuild the two CSVs from the Superstore export
jupyter nbconvert --to notebook --execute --inplace notebooks/delta_scd_assignment.ipynb
python scripts/capture_screenshots.py   # re-render screenshots/ from the executed notebook
python scripts/build_report.py       # rebuild this PDF
```

The notebook is idempotent — RESET_LAKE = True drops and rebuilds every Delta table on each run, so re-executing it always produces the numbers in this report.